

Mini Paper — 1.3 Exchanging Data

Time: ~35 minutes · Total: 30 marks · Answer all questions.

Mark your work against the mark scheme at the end; aim for ~1 minute per mark.

Questions

Q1. Explain what is meant by *hashing* and give **two** reasons why hashing is used to store passwords rather than storing the passwords directly. **[3]** (AO1)

Q2. A company sends confidential data over the internet.

(a) State the difference between *symmetric* and *asymmetric* encryption. **[2]** (AO1)

(b) Give **one** advantage of using asymmetric encryption when two parties have never communicated before. **[1]** (AO1)

(c) Explain why the company hashes its stored user passwords but encrypts the data it transmits, referring to the key difference between hashing and encryption. **[1]** (AO2)

Q3. A row of pixels in a monochrome image is shown below, where each letter is a pixel colour:

BBBBBBBWWWWBBBBBB

(a) Show how *run length encoding* (RLE) would encode this row. **[2]** (AO2)

(b) State **one** type of data for which RLE would be a poor choice, and explain why. **[1]** (AO2)

Q4. A flat-file table storing patient appointments is being normalised.

Appointment(AppointmentID, PatientID, PatientName, DoctorID, DoctorName, RoomNo)

Here DoctorName and RoomNo are determined by DoctorID.

(a) State the rule a table must satisfy to be in *first normal form* (1NF). **[1]** (AO1)

(b) The table is already in 2NF. Explain why it is **not** in *third normal form* (3NF), and show the tables that result from normalising it to 3NF. **[3]** (AO2)

Q5. A Member table has the fields MemberID, Surname, Town and JoinYear.

Write an SQL statement to display the MemberID and Surname of all members whose Town is 'Leeds' and who have a JoinYear before 2020, sorted by Surname in descending order. **[4]** (AO2)

Q6. Describe the role of the **Transport** layer and the **Network/Internet** layer of the TCP/IP stack when data is sent. **[3]** (AO1)

Q7. Compare *packet switching* and *circuit switching*. **[3]** (AO2)

Q8. An online banking website lets customers log in and transfer money between accounts.

Discuss how *client-side processing* and *server-side processing* should be used for this website, and explain the role of **encryption** and **transaction processing (ACID)** in keeping it reliable and secure. Make a justified recommendation. **[6]** (AO3)

Mark scheme

Q1 — [3] (AO1). Indicative content (1 mark each, max 3): - Hashing applies a hash function to an input to produce a fixed-length output (hash/digest) (1). - It is one-way — the hash cannot feasibly be reversed back to the original password (1). - If the database is stolen, attackers do not obtain the actual passwords (1). - At login the entered password is hashed and compared to the stored hash (1).

Examiner tip: state that hashing is one-way — this is the property that makes stored hashes safer than stored plaintext.

Q2 — [4] total. (a) [2] (AO1): - Symmetric uses the **same single key** to encrypt and decrypt (1). - Asymmetric uses a **pair** of keys — a public key and a related private key (1).

(b) [1] (AO1): - It solves the key distribution problem / the public key can be shared openly so no secret key needs to be exchanged beforehand (1).

(c) [1] (AO2): - Encryption is reversible (with a key) so transmitted data can be decrypted by the recipient, whereas hashing is one-way so stored passwords cannot be recovered even by the company (1).

Examiner tip: be precise — "same key" for symmetric, "public + private pair" for asymmetric; don't just say "two keys".

Q3 — [3] total. (a) [2] (AO2): - Correct counts and values, e.g. 7B 4W 6B / 7B4W6B (1). - Encoding correctly given as value + run length pairs covering the whole row (1).

(b) [1] (AO2): - Data with few/no repeated runs (e.g. text, a photographic/complex image); RLE can make the file larger because each value also stores a count (1).

Examiner tip: write the runs in order and check they sum back to the original length (7+4+6 = 17).

Q4 — [4] total. (a) [1] (AO1): - No repeating groups; each field holds a single (atomic) value; the table has a primary key (1).

(b) [3] (AO2): - It is not in 3NF because DoctorName / RoomNo depend on DoctorID, a non-key attribute — a transitive (non-key) dependency (1). - Split out the doctor data into its own table (1): -

Doctor(DoctorID, DoctorName, RoomNo) - Appointment(AppointmentID, PatientID, DoctorID*) with DoctorID as a foreign key (1). - (Allow Patient(PatientID, PatientName) also split out as a valid extra refinement.)

Examiner tip: name the dependency you are removing (transitive → 3NF) and show the foreign key that links the new tables.

Q5 — [4] (AO2).

```
SELECT MemberID, Surname
FROM Member
WHERE Town = 'Leeds' AND JoinYear < 2020
ORDER BY Surname DESC;
```

- Correct `SELECT MemberID, Surname` (1).
- Correct `FROM Member` (1).
- Correct `WHERE Town = 'Leeds' AND JoinYear < 2020` (1).
- Correct `ORDER BY Surname DESC` (1).

Examiner tip: keep clause order SELECT → FROM → WHERE → ORDER BY, and remember DESC for descending.

Q6 — [3] (AO1). Max 3 from: - Transport: splits data into packets / manages end-to-end delivery and reassembly (1). - Transport: adds port numbers and handles error checking (TCP) (1). - Network/Internet: adds source and destination IP addresses (1). - Network/Internet: routes the packets across networks (1).

Examiner tip: don't confuse port numbers (Transport) with IP addresses (Network) — each layer adds its own header.

Q7 — [3] (AO2). Max 3 comparison points: - Circuit switching reserves a dedicated path for the whole communication; packet switching does not (1). - In packet switching data is split into packets routed independently / by different routes and reassembled at the destination (1). - Packet switching uses the network more efficiently — no bandwidth wasted when idle / it is more resilient and can route around failures (1). - Circuit switching guarantees constant bandwidth/connection for the duration (1).

Examiner tip: make genuine comparisons (X does ... whereas Y does ...), not two separate lists.

Q8 — [6] (AO3). Levelled response; indicative content:

AO1 (knowledge): - Client-side runs in the user's browser (e.g. JavaScript); server-side runs on the web server. - Encryption (HTTPS) protects data in transit; ACID = Atomicity, Consistency, Isolation, Durability.

AO2 (application): - Client-side validation (e.g. checking fields are filled/numeric) gives instant feedback and reduces wasted server requests. - Server-side processing performs the actual login check and money transfer against the database, hidden from the user. - A transfer is a transaction: Atomicity ensures the debit and credit both happen or neither does; Durability ensures a committed transfer survives a power failure; Isolation/record locking stops concurrent transfers corrupting balances.

AO3 (evaluation/judgement): - Client-side validation can be viewed, disabled or bypassed, so it is not secure on its own; security-critical checks (login, transfer amount, balance) must be repeated server-side. - HTTPS should encrypt all data in transit (asymmetric to exchange a session key, then symmetric for bulk data); passwords stored as hashes. - Justified recommendation: use client-side for responsiveness but rely on server-side processing, encryption and ACID transactions for security and reliability — best practice combines both.

Banding: - **5–6:** balanced, well applied to the banking scenario, covers both processing types plus encryption and ACID, with a justified recommendation. - **3–4:** some application and comparison; encryption or ACID may be thin; conclusion limited. - **1–2:** basic descriptive points, little application or judgement.

Examiner tip: for AO3 marks you must weigh both sides and finish with a clear, justified recommendation — don't just describe each in turn.